Candlestick.cpp:

```
// ======================================================
// START OF MY OWN CODE
// ======================================================
#include "Candlestick.h"
// No extra implementation needed for now.
// ======================================================
// END OF MY OWN CODE
// ======================================================
```

Candlestick.h:

```
// ======================================================
// START OF MY OWN CODE
// ======================================================
#pragma once
#include <string>

struct Candlestick {
  double open;
  double high;
  double low;
  double close;
  std::string startTime;
  std::string endTime;

  Candlestick() {}

  Candlestick(double o,
              double h,
              double l,
              double c,
              const std::string& start,
              const std::string& end)
      : open(o), high(h), low(l), close(c),
        startTime(start), endTime(end) {}
};

// ======================================================
// END OF MY OWN CODE
// ======================================================
```

CandlestickPlot.cpp:

```
// ========================================================
// START OF MY OWN CODE
// ========================================================

#include "CandlestickPlot.h"
#include <iostream>
#include <iomanip>
#include <algorithm>

void plotCandlesticksASCII(const std::vector<Candlestick>& candles,
                           std::size_t maxToShow)
{
    if (candles.empty()) {
        std::cout << "No candlestick data available.\n";
        return;
    }

    std::size_t count = candles.size();
    std::size_t startIndex = 0;

    if (count > maxToShow) {
        startIndex = count - maxToShow;
    }

    // ----------------------------------------------------
    // Find overall min/max for scaling
    // ----------------------------------------------------
    double globalMin = candles[startIndex].low;
    double globalMax = candles[startIndex].high;

    for (std::size_t i = startIndex; i < count; ++i) {
        globalMin = std::min(globalMin, candles[i].low);
        globalMax = std::max(globalMax, candles[i].high);
    }

    double range = globalMax - globalMin;
    if (range <= 0.0) range = 1.0;

    // ----------------------------------------------------
    // Print unified OHLC summary table
    // ----------------------------------------------------
    std::cout << "\n===== OHLC SUMMARY TABLE =====\n";
```

```
    std::cout << "DATE       | "
              << "OPEN       HIGH       LOW        CLOSE\n";
    std::cout << "-------------------------------------------------------------\n";

    std::cout << std::fixed << std::setprecision(2);

    for (std::size_t i = startIndex; i < count; ++i) {
        const auto& c = candles[i];
        std::string label = c.startTime.substr(0, 10);

        std::cout << label << " | "
                  << "OPEN: "  << std::setw(8) << c.open  << "  "
                  << "HIGH: "  << std::setw(8) << c.high  << "  "
                  << "LOW: "   << std::setw(8) << c.low   << "  "
                  << "CLOSE: " << std::setw(8) << c.close
                  << "\n";
    }

// ========================================================
// END OF MY OWN CODE
// ========================================================
```

CandlestickPlot.h:

```
// ========================================================
// START OF MY OWN CODE
// This header file was written entirely by me and was not
// provided as part of the course template.
// ========================================================

#pragma once
#include <vector>
#include "Candlestick.h"

void plotCandlesticksASCII(const std::vector<Candlestick>& candles,
                           std::size_t maxToShow = 20);

// ========================================================
// END OF MY OWN CODE
// ========================================================
```

CSVReader.cpp:

```
#include "CSVReader.h"
#include <iostream>
#include <fstream>
#include <sstream>

// ============================================================
// START OF MY OWN CODE
// ============================================================

std::vector<OrderBookEntry> CSVReader::readCSV(std::string filename)
{
    std::vector<OrderBookEntry> entries;
    std::ifstream file{ filename };
    std::string line;

    if (!file.is_open()) {
        std::cout << "Failed to open file: " << filename << std::endl;
        return entries;
    }

    // Skip header row
    std::getline(file, line);

    while (std::getline(file, line))
    {
        if (line.size() == 0)
            continue;

        std::vector<std::string> tokens = tokenize(line, ',');

        // Expected format:
        // timestamp, product, orderType, price, amount
        if (tokens.size() != 5)
            continue;

        try {
            double price  = std::stod(tokens[3]);
            double amount = std::stod(tokens[4]);

            OrderType type;
            if (tokens[2] == "bid")
                type = OrderType::bid;
```

```
            else if (tokens[2] == "ask")
                type = OrderType::ask;
            else
                continue;

            OrderBookEntry entry{
                price,
                amount,
                tokens[0],  // timestamp
                tokens[1],  // product
                type,
                "dataset"   // default username
            };

            entries.push_back(entry);
        }
        catch (...) {
            // Ignore malformed rows
            continue;
        }
    }

    std::cout << "CSVReader::readCSV read "
              << entries.size()
              << " entries" << std::endl;

    return entries;
}

std::vector<std::string> CSVReader::tokenize(std::string line, char delimiter)
{
    std::vector<std::string> tokens;
    std::string token;
    std::stringstream ss(line);

    while (std::getline(ss, token, delimiter)) {
        tokens.push_back(token);
    }

    return tokens;
}
```

// ========== END OF MY OWN CODE ==========

CSVReader.h:

```
#pragma once
#include <vector>
#include <string>
#include "OrderBookEntry.h"

class CSVReader {
public:
  static std::vector<OrderBookEntry> readCSV(std::string filename);
  static std::vector<std::string> tokenize(std::string line, char delimiter);
};
```

main.cpp:

```
#include "MerkelMain.h"

int main() {
  MerkelMain app;
  app.init();
  return 0;
}
```

MerkelMain.cpp:

```
#include "MerkelMain.h"
#include "OrderBookEntry.h"
#include "Wallet.h"


#include <iostream>
#include <fstream>
#include <limits>
#include <iomanip>
#include <map>
#include <cmath>
#include <algorithm>
#include <vector>
#include <string>
#include <chrono>
#include <sstream>
```

```
void printOHLCtable(const std::vector<Candlestick>& candles);
std::string getSystemTimestamp()
{
    using namespace std::chrono;
    auto now = system_clock::now();
    auto s = time_point_cast<seconds>(now);
    std::time_t t = system_clock::to_time_t(s);

    std::tm tm{};
#if defined(_WIN32)
    localtime_s(&tm, &t);
#else
    localtime_r(&t, &tm);
#endif

    char buf[20];
    std::strftime(buf, sizeof(buf), "%Y/%m/%d %H:%M", &tm);
    return std::string(buf);
}

static std::string nowTimestamp()
{
    using namespace std::chrono;

    auto now = system_clock::now();
    auto ms = duration_cast<milliseconds>(now.time_since_epoch()) % 1000;

    std::time_t t = system_clock::to_time_t(now);
    std::tm tm{};
#if defined(_WIN32) || defined(_WIN64)
    localtime_s(&tm, &t);
#else
    localtime_r(&t, &tm);
#endif

    char buf[32];
    std::strftime(buf, sizeof(buf), "%Y/%m/%d %H:%M:%S", &tm);

    std::string out(buf);
    out += ".";
    out += std::to_string((int)ms.count()); // milliseconds
    return out;
}
```

```
// ============================================================
// START OF MY OWN CODE
// ============================================================

static void ensureTransactionsHeader()
{
   std::ifstream in("transactions.csv");
   if (!in.good())
   {
      std::ofstream out("transactions.csv");
      out << "username,type,product,price,amount,timestamp\n";
      return;
   }

   in.seekg(0, std::ios::end);
   if (in.tellg() == 0)
   {
      std::ofstream out("transactions.csv", std::ios::app);
      out << "username,type,product,price,amount,timestamp\n";
   }
}

static void appendTransactionCSV(const std::string& username,
                                 const std::string& type,
                                 const std::string& product,
                                 double price,
                                 double amount,
                                 const std::string& timestamp)
{
   ensureTransactionsHeader();

   std::ofstream file("transactions.csv", std::ios::app);
   if (!file.is_open()) return;

   file << username << ","
        << type << ","
        << product << ","
        << price << ","
        << amount << ","
        << timestamp
        << "\n";
}
```

```
// ============================================================
// END OF MY OWN CODE
// ============================================================


MerkelMain::MerkelMain()
: orderBook{"20200601.csv"}
{

}

void MerkelMain::init() {
    currentTime = orderBook.getEarliestTime();
    std::cout << "Current timestamp: " << currentTime << "\n\n";
    mainLoop();
}

// ============================================================
// Menu
// ============================================================
void MerkelMain::printMenu() {
    std::cout << "\n========== ACCOUNT MENU ==========\n";
    std::cout << " 100: Register new user\n";
    std::cout << " 101: Login\n";
    std::cout << " 102: Reset password\n";

    std::cout << "\n========== MAIN MENU ==========\n";
    std::cout << " 1 : Make an ask (sell)\n";
    std::cout << " 2 : Make a bid (buy)\n";
    std::cout << " 3 : Move to next timeframe\n";
    std::cout << " 4 : Market stats\n";
    std::cout << " 5 : Print wallet\n";
    std::cout << " 6 : Deposit money\n";
    std::cout << " 7 : Withdraw money\n";
    std::cout << " 8 : Transaction history\n";
    std::cout << " 9 : Candlesticks\n";
    std::cout << " 10: Simulate trades\n";
    std::cout << " 11: User activity summary\n";
    std::cout << " 0 : Exit\n";
    std::cout << "================================\n";
}
```

```
void MerkelMain::mainLoop() {
    bool running = true;

    while (running) {
        printMenu();

        int choice;
        std::cin >> choice;

        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(9999, '\n');
            std::cout << "Invalid input.\n";
            continue;
        }

        switch (choice) {

// ========================================================
// START OF MY OWN CODE
// ========================================================

        case 100: {
            std::string fullname, email, password;
            std::cout << "Enter full name: ";
            std::getline(std::cin >> std::ws, fullname);

            std::cout << "Enter email: ";
            std::getline(std::cin >> std::ws, email);

            std::cout << "Enter password: ";
            std::getline(std::cin >> std::ws, password);

            if (userManager.registerUser(fullname, email, password)) {
                userLoggedIn = true;
                wallet = Wallet();
                std::string username =
                    userManager.getLoggedInUser().username;

                wallet.loadFromCSV("wallet.csv", username);
                std::cout << "Logged in as: " << username << "\n";
            }
```

```
    break;
}

case 101: {
    std::string inputUsername, password;

    std::cout << "Enter username: ";
    std::getline(std::cin >> std::ws, inputUsername);

    std::cout << "Enter password: ";
    std::getline(std::cin >> std::ws, password);

    if (userManager.loginUser(inputUsername, password))
    {
        userLoggedIn = true;
        wallet = Wallet();
        std::string loggedUser = userManager.getLoggedInUser().username;

        if (wallet.existsInCSV("wallet.csv", loggedUser))
            wallet.loadFromCSV("wallet.csv", loggedUser);
        else
        {
            wallet.insertCurrency("BTC", 10);
            wallet.insertCurrency("USDT", 1000);
            wallet.saveToCSV("wallet.csv", loggedUser);
        }

        std::cout << "Logged in as: " << loggedUser << "\n";
    }
    break;
}

case 102: {
    auto usernames = userManager.getAllUsernames();

    if (usernames.empty()) {
        std::cout << "No users found.\n";
        break;
    }

    std::cout << "\n=== SELECT USER TO RESET PASSWORD ===\n";
    for (int i = 0; i < (int)usernames.size(); ++i)
        std::cout << i << ") " << usernames[i] << "\n";
```

```
            int idx;
            std::cout << "Choose user index: ";
            std::cin >> idx;

            if (idx < 0 || idx >= (int)usernames.size()) {
                std::cout << "Invalid selection.\n";
                break;
            }

            std::string email, newPassword;
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

            std::cout << "Enter email (must be @gmail.com): ";
            std::getline(std::cin, email);

            std::cout << "Enter new password: ";
            std::getline(std::cin, newPassword);

            userManager.resetPassword(usernames[idx], email, newPassword);
            break;
        }

// ============================================================
// END OF MY OWN CODE
// ============================================================

        default:

            if (!userLoggedIn && choice != 0) {
                std::cout << "You must log in first.\n";
                break;
            }
            else if (choice == 1) enterAsk();
            else if (choice == 2) enterBid();
            else if (choice == 3) gotoNextTimeFrame();
            else if (choice == 4) printMarketStats();
            else if (choice == 5) printWallet();
            else if (choice == 6) depositMoney();
            else if (choice == 7) withdrawMoney();
            else if (choice == 8) showTransactionHistory();
            else if (choice == 9) handleCandlestickTimeframe("");
            else if (choice == 10) simulateTradingActivity();
            else if (choice == 11) showUserActivitySummary();
```

```
        else if (choice == 0) {
            std::cout << "Exiting...\n";
            running = false;
        }
        break;
      }
   }
}


// ========================================================
// START OF MY OWN CODE
// ========================================================

void MerkelMain::printWallet() {
   std::cout << wallet.toString() << "\n";
}

// ========================================================
// Market stats
// ========================================================
void MerkelMain::printMarketStats()
{
  std::cout << "\n=== MARKET STATS (" << currentTime << ") ===\n";

  for (const std::string& product : orderBook.getKnownProducts())
  {
     std::cout << "Product: " << product << "\n";

     std::vector<OrderBookEntry> asks = orderBook.getOrders(product, currentTime, OrderType::ask);
     std::cout << "Asks seen: " << asks.size() << "\n";

     if (!asks.empty())
     {
        double maxAsk = asks[0].price;
        double minAsk = asks[0].price;
        for (const auto& a : asks)
        {
           maxAsk = std::max(maxAsk, a.price);
           minAsk = std::min(minAsk, a.price);
        }
        std::cout << "Max ask: " << maxAsk << "\n";
        std::cout << "Min ask: " << minAsk << "\n";
     }
```

```
      else
      {
         std::cout << "Max ask: N/A\n";
         std::cout << "Min ask: N/A\n";
      }

      std::cout << "\n";
   }
}

// ============================================================
// Timeframe progression + trade execution
// ============================================================
void MerkelMain::gotoNextTimeFrame()
{
   std::cout << "\nAdvancing to next timeframe...\n";

   const std::string currentUser = userManager.getLoggedInUser().username;

   // First, execute any matches in the CURRENT timeframe.
   std::vector<OrderBookEntry> executedForUser;
   for (const std::string& product : orderBook.getKnownProducts())
   {
      std::vector<OrderBookEntry> sales = orderBook.matchAsksToBids(product, currentTime);

      for (const auto& sale : sales)
      {
         if (sale.username != currentUser)
            continue;

         wallet.processSale(sale);
         executedForUser.push_back(sale);

         if (sale.orderType == OrderType::asksale)
            appendTransactionCSV(currentUser, "ASK_SALE", product, sale.price, sale.amount,
currentTime);
         else if (sale.orderType == OrderType::bidsale)
            appendTransactionCSV(currentUser, "BID_SALE", product, sale.price, sale.amount,
currentTime);
      }
   }

   // Show what was actually processed
   if (executedForUser.empty())
```

```
    {
        std::cout << "No matched trades were executed for your orders in this timeframe.\n";
    }
    else
    {
        std::cout << "Executed trades:\n";
        std::cout << std::left
                  << std::setw(8)  << "Side"
                  << std::setw(12) << "Product"
                  << std::setw(12) << "Price"
                  << std::setw(12) << "Amount"
                  << "Timestamp" << "\n";
        std::cout << std::string(60, '-') << "\n";

        for (const auto& s : executedForUser)
        {
            const char* side = (s.orderType == OrderType::bidsale) ? "BUY" : "SELL";
            std::cout << std::left
                      << std::setw(8)  << side
                      << std::setw(12) << s.product
                      << std::setw(12) << s.price
                      << std::setw(12) << s.amount
                      << s.timestamp
                      << "\n";
        }
        std::cout << std::string(60, '-') << "\n";
    }

    // Persist wallet after processing.
    wallet.saveToCSV("wallet.csv", currentUser);

    // Now advance the market time.
    currentTime = orderBook.getNextTime(currentTime);
    std::cout << "Current timestamp: " << currentTime << "\n";
}

// ============================================================
// Ask & Bid
// ============================================================
void MerkelMain::enterAsk()
{
    auto products = orderBook.getKnownProducts();

    std::cout << "\n=== AVAILABLE PRODUCTS ===\n";
```

```
 for (const auto& p : products)
    std::cout << "  " << p << "\n";

 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

 std::string product;
 std::cout << "Enter product to SELL (e.g. BTC/USDT): ";
 std::getline(std::cin, product);

 if (std::find(products.begin(), products.end(), product) == products.end())
 {
    std::cout << "Invalid product.\n";
    return;
 }

 double price, amount;

 std::cout << "Price: ";
 std::cin >> price;

 std::cout << "Amount: ";
 std::cin >> amount;

 // market timeframe, and must be wallet-fulfillable before being placed.
 OrderBookEntry order(
    price,
    amount,
    currentTime,
    product,
    OrderType::ask,
    userManager.getLoggedInUser().username
 );

 if (!wallet.canFulfillOrder(order))
 {
    std::cout << "Wallet has insufficient funds for this ask.\n";
    return;
 }

 orderBook.insertOrder(order);

appendTransactionCSV(
 userManager.getLoggedInUser().username,
 "ASK",
```

```
    product,
    price,
    amount,
    currentTime
);

    std::cout << "Ask submitted for " << product << ".\n";
}


void MerkelMain::enterBid()
{
    auto products = orderBook.getKnownProducts();

    std::cout << "\n=== AVAILABLE PRODUCTS ===\n";
    for (const auto& p : products)
        std::cout << "  " << p << "\n";

    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    std::string product;
    std::cout << "Enter product to BUY (e.g. BTC/USDT): ";
    std::getline(std::cin, product);

    if (std::find(products.begin(), products.end(), product) == products.end())
    {
        std::cout << "Invalid product.\n";
        return;
    }

    double price, amount;

    std::cout << "Price: ";
    std::cin >> price;

    std::cout << "Amount: ";
    std::cin >> amount;

    OrderBookEntry order(
        price,
        amount,
        currentTime,
        product,
        OrderType::bid,
```

```
        userManager.getLoggedInUser().username
    );

    if (!wallet.canFulfillOrder(order))
    {
        std::cout << "Wallet has insufficient funds for this bid.\n";
        return;
    }

    orderBook.insertOrder(order);

   appendTransactionCSV(
    userManager.getLoggedInUser().username,
    "BID",
    product,
    price,
    amount,
    currentTime
);

    std::cout << "Bid submitted for " << product << ".\n";
}

void MerkelMain::depositMoney() {
    double amount;
    std::cout << "Deposit USDT amount: ";
    std::cin >> amount;

    if (amount > 0) {
        wallet.insertCurrency("USDT", amount);
        wallet.saveToCSV("wallet.csv",
            userManager.getLoggedInUser().username);

        appendTransactionCSV(
    userManager.getLoggedInUser().username,
    "DEPOSIT",
    "USDT",
    1,
    amount,
    currentTime
);


        std::cout << "Deposit successful.\n";
```

```
    }
}


void MerkelMain::withdrawMoney() {
    double amount;
    std::cout << "Withdraw USDT amount: ";
    std::cin >> amount;

    if (!wallet.containsCurrency("USDT", amount)) {
        std::cout << "Insufficient funds.\n";
        return;
    }

    wallet.removeCurrency("USDT", amount);
    wallet.saveToCSV("wallet.csv",
        userManager.getLoggedInUser().username);

    appendTransactionCSV(
    userManager.getLoggedInUser().username,
    "WITHDRAW",
    "USDT",
    1,
    amount,
    currentTime
);


    std::cout << "Withdrawal successful.\n";
}


// ============================================================
// Print Products
// ============================================================
void MerkelMain::printProducts() {
    auto products = orderBook.getKnownProducts();
    for (const auto& p : products)
        std::cout << "  " << p << "\n";
}

void MerkelMain::showTransactionHistory()
{
    const std::string user = userManager.getLoggedInUser().username;
```

```
struct Tx {
    std::string when;
    std::string type;
    std::string product;
    double price = 0.0;
    double amount = 0.0;
};

auto split = [](const std::string& s) {
    std::vector<std::string> parts;
    std::string cur;
    for (char c : s) {
        if (c == ',') { parts.push_back(cur); cur.clear(); }
        else cur.push_back(c);
    }
    parts.push_back(cur);
    return parts;
};

std::ifstream in("transactions.csv");
if (!in.is_open()) {
    std::cout << "No transaction history file found.\n";
    return;
}

std::string line;
std::getline(in, line); // header

std::vector<Tx> all;
while (std::getline(in, line)) {
    if (line.rfind(user + ",", 0) != 0)
        continue;

    auto c = split(line);
    // username,type,product,price,amount,timestamp
    if (c.size() < 6)
        continue;

    Tx t;
    t.type = c[1];
    t.product = c[2];
    t.when = c[5];
    try { t.price = std::stod(c[3]); } catch (...) { t.price = 0.0; }
```

```
    try { t.amount = std::stod(c[4]); } catch (...) { t.amount = 0.0; }
    all.push_back(t);
}

if (all.empty()) {
    std::cout << "No transactions recorded yet.\n";
    return;
}

auto printList = [&](const std::vector<Tx>& rows) {
    std::cout << "\n--- Transactions ---\n";
    std::cout << std::left
              << std::setw(20) << "Timestamp"
              << std::setw(10) << "Type"
              << std::setw(12) << "Product"
              << std::setw(12) << "Price"
              << std::setw(12) << "Amount"
              << "\n";
    std::cout << std::string(66, '-') << "\n";

    for (const auto& t : rows) {
        std::cout << std::left
                  << std::setw(20) << t.when
                  << std::setw(10) << t.type
                  << std::setw(12) << t.product
                  << std::setw(12) << t.price
                  << std::setw(12) << t.amount
                  << "\n";
    }
    std::cout << std::string(66, '-') << "\n";
};

std::cout << "\n=== Transaction History ===\n";
std::cout << " 1) Show last 5\n";
std::cout << " 2) Filter by product\n";
std::cout << "Choice: ";

int choice = 0;
std::cin >> choice;
if (std::cin.fail()) {
    std::cin.clear();
    std::cin.ignore(9999, '\n');
    std::cout << "Invalid input.\n";
    return;
```

```
    }

    std::vector<Tx> view;
    if (choice == 1) {
        int start = std::max(0, (int)all.size() - 5);
        for (int i = (int)all.size() - 1; i >= start; --i)
            view.push_back(all[i]);
        printList(view);
        return;
    }

    if (choice == 2) {
        auto products = orderBook.getKnownProducts();

        // clear the newline left by std::cin >> choice
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        std::cout << "Available products:\n";
        for (const auto& p : products)
            std::cout << "  " << p << "\n";

        std::string product;
        std::cout << "Enter product: ";
        std::getline(std::cin, product);

        for (int i = (int)all.size() - 1; i >= 0; --i)
            if (all[i].product == product)
                view.push_back(all[i]);

        if (view.empty())
            std::cout << "No transactions found for product: " << product << "\n";
        else
            printList(view);
        return;
    }

    std::cout << "Invalid choice.\n";
}

std::vector<Candlestick>
MerkelMain::buildMinuteCandlesticks(const std::string& product)
{
    std::vector<OrderBookEntry> all =
        orderBook.getAllOrdersForProduct(product);
```

```
    auto txOrders = getTransactionOrdersForProduct(product);
    all.insert(all.end(), txOrders.begin(), txOrders.end());

    if (all.empty()) return {};
    std::stable_sort(
        all.begin(),
        all.end(),
        [](const OrderBookEntry& a, const OrderBookEntry& b)
        {
            return a.timestamp < b.timestamp;
        }
    );


    std::map<std::string, std::vector<OrderBookEntry>> grouped;

    for (auto& o : all) {
        std::string key = o.timestamp.substr(0, 16); // YYYY/MM/DD HH:MM
        grouped[key].push_back(o);
    }

    std::vector<Candlestick> candles;

    for (auto& pair : grouped) {
        auto& entries = pair.second;
        if (entries.empty()) continue;

        double open  = entries.front().price;
        double close = entries.back().price;
        double high  = open;
        double low   = open;

        for (auto& e : entries) {
            high = std::max(high, e.price);
            low  = std::min(low,  e.price);
        }

        candles.push_back(Candlestick(open, high, low, close,
                                      pair.first, pair.first));
    }

    return candles;
}
```

```
std::vector<OrderBookEntry>
MerkelMain::getTransactionOrdersForProduct(const std::string& product)
{
  std::vector<OrderBookEntry> out;
  std::ifstream file("transactions.csv");
  if (!file.is_open())
     return out;

  std::string line;
  std::getline(file, line); // skip header

  while (std::getline(file, line))
  {
     std::stringstream ss(line);
     std::string user, type, prod, priceStr, amountStr, timestamp;

     std::getline(ss, user, ',');
     std::getline(ss, type, ',');
     std::getline(ss, prod, ',');
     std::getline(ss, priceStr, ',');
     std::getline(ss, amountStr, ',');
     std::getline(ss, timestamp, ',');

     if (prod != product) continue;
     if (type != "ASK" && type != "BID") continue;

     double price = std::stod(priceStr);
     double amount = std::stod(amountStr);

     out.emplace_back(
        price,
        amount,
        timestamp,
        product,
        (type == "ASK") ? OrderType::ask : OrderType::bid,
        user
     );
  }

  return out;
}

// ====================================================
// Timeframe Handler
```

```
// ============================================================
void MerkelMain::handleCandlestickTimeframe(const std::string&)
{
  auto products = orderBook.getKnownProducts();
  if (products.empty())
  {
     std::cout << "No products available.\n";
     return;
  }

  // ----------------------------
  // SHOW PRODUCT LIST
  // ----------------------------
  std::cout << "\n=== AVAILABLE PRODUCTS ===\n";
  for (const auto& p : products)
     std::cout << "  " << p << "\n";

  std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

  std::string product;
  std::cout << "Enter product (e.g. BTC/USDT): ";
  std::getline(std::cin, product);

  if (std::find(products.begin(), products.end(), product) == products.end())
  {
     std::cout << "Invalid product.\n";
     return;
  }

  auto orders = orderBook.getAllOrdersForProduct(product);
  if (orders.empty())
  {
     std::cout << "No data for product.\n";
     return;
  }

  // ============================================================
  // DEFAULT: YEARLY SUMMARY
  // ============================================================
  std::map<std::string, std::vector<OrderBookEntry>> askBuckets, bidBuckets;

for (const auto& o : orderBook.getAllOrdersForProduct(product))
{
  std::string yearKey = o.timestamp.substr(0, 4);
```

```
  if (o.orderType == OrderType::ask)
     askBuckets[yearKey].push_back(o);
  else if (o.orderType == OrderType::bid)
     bidBuckets[yearKey].push_back(o);
}

auto txOrders = getTransactionOrdersForProduct(product);

for (auto o : txOrders)
{
  o.timestamp = "2026" + o.timestamp.substr(4);

  if (o.orderType == OrderType::ask)
     askBuckets["2026"].push_back(o);
  else if (o.orderType == OrderType::bid)
     bidBuckets["2026"].push_back(o);
}



  auto buildCandles = [](auto& buckets)
  {
     std::vector<Candlestick> out;

     for (auto& pair : buckets)
     {
        auto& v = pair.second;
        std::stable_sort(v.begin(), v.end(),
            [](const auto& a, const auto& b) {
               return a.timestamp < b.timestamp;
            });


        double open = v.front().price;
        double close = v.back().price;
        double high = open, low = open;

        for (auto& e : v)
        {
           high = std::max(high, e.price);
           low  = std::min(low,  e.price);
        }
```

```
        out.emplace_back(open, high, low, close,
                         pair.first, pair.first);
    }
    return out;
};

auto askYearly = buildCandles(askBuckets);
auto bidYearly = buildCandles(bidBuckets);

std::cout << "\n(YEARLY summary)\n";

std::cout << "\nProduct: " << product << " | ASKS | YEARLY\n";
printOHLCtable(askYearly);

std::cout << "\nProduct: " << product << " | BIDS | YEARLY\n";
printOHLCtable(bidYearly);

// ================================================
// OPTIONAL FILTER
// ================================================
std::cout << "\nView another range?\n";
std::cout << "1) Daily (minute candlesticks)\n";
std::cout << "2) Monthly\n";
std::cout << "0) Exit\n";
std::cout << "Choice: ";

int choice;
std::cin >> choice;

if (choice == 0) return;

// ================================================
// DAILY → MINUTE CANDLESTICKS
// ================================================
if (choice == 1)
{
    auto candles = buildMinuteCandlesticks(product);
    if (candles.empty())
    {
        std::cout << "No minute data available.\n";
        return;
    }

    int minutes;
```

```
        std::cout << "How many previous minutes? ";
        std::cin >> minutes;

        if (std::cin.fail() || minutes <= 0)
        {
            std::cout << "Invalid number.\n";
            return;
        }

        int total = candles.size();
        int start = std::max(0, total - minutes);

        printMinuteCandlestickTable(candles, start, total - 1);
        return;
    }

    // ========================================================
    // MONTHLY SUMMARY
    // ========================================================
    if (choice == 2)
    {
        askBuckets.clear();
        bidBuckets.clear();

        for (const auto& o : orders)
        {
            std::string monthKey = o.timestamp.substr(0, 7); // YYYY/MM

            if (o.orderType == OrderType::ask)
                askBuckets[monthKey].push_back(o);
            else if (o.orderType == OrderType::bid)
                bidBuckets[monthKey].push_back(o);
        }

        auto askMonthly = buildCandles(askBuckets);
        auto bidMonthly = buildCandles(bidBuckets);

        std::cout << "\nProduct: " << product << " | ASKS | MONTHLY\n";
        printOHLCtable(askMonthly);

        std::cout << "\nProduct: " << product << " | BIDS | MONTHLY\n";
        printOHLCtable(bidMonthly);
    }
}
```

```
// ============================================================
// Print Table
// ============================================================
void MerkelMain::printMinuteCandlestickTable(const std::vector<Candlestick>& candles,
                                             int startIndex, int endIndex)
{
  std::cout << "\n==================== MINUTE CANDLES ====================\n\n";

std::cout << std::left
        << std::setw(20) << "TIME"
        << std::setw(18) << "OPEN"
        << std::setw(18) << "HIGH"
        << std::setw(18) << "LOW"
        << std::setw(18) << "CLOSE"
        << "\n-------------------------------------------------------------------------------\n";

std::cout << std::fixed << std::setprecision(6);  // ← SET ONCE

for (int i = startIndex; i <= endIndex; ++i)
{
  const Candlestick& c = candles[i];

  std::cout << std::left
            << std::setw(20) << c.startTime
            << std::setw(18) << c.open
            << std::setw(18) << c.high
            << std::setw(18) << c.low
            << std::setw(18) << c.close
            << "\n";
}

std::cout << "-------------------------------------------------------------------------------\n";

}


// ============================================================
// Simulation
// ============================================================
void MerkelMain::simulateTradingActivity()
{
```

```
std::cout << "\n--- Simulating Trading Activity ---\n";
std::cout << "This will create 5 ASKs + 5 BIDs for EACH product using dataset time.\n";

const std::string currentUser = userManager.getLoggedInUser().username;
auto products = orderBook.getKnownProducts();

for (const auto& product : products)
{
   std::cout << "\nProduct: " << product << "\n";

   // -------------------------------------------------
   // MARKET SNAPSHOT (DATASET TIME)
   // -------------------------------------------------
   auto asks = orderBook.getOrders(product, currentTime, OrderType::ask);
   auto bids = orderBook.getOrders(product, currentTime, OrderType::bid);

   double minAsk = 0.0, maxAsk = 0.0;
   double minBid = 0.0, maxBid = 0.0;

   if (!asks.empty())
   {
      minAsk = maxAsk = asks[0].price;
      for (const auto& a : asks)
      {
         minAsk = std::min(minAsk, a.price);
         maxAsk = std::max(maxAsk, a.price);
      }
   }

   if (!bids.empty())
   {
      minBid = maxBid = bids[0].price;
      for (const auto& b : bids)
      {
         minBid = std::min(minBid, b.price);
         maxBid = std::max(maxBid, b.price);
      }
   }

   std::cout << "Market snapshot:\n";
   std::cout << "  Ask range : " << minAsk << " → " << maxAsk << "\n";
   std::cout << "  Bid range : " << minBid << " → " << maxBid << "\n";

   // -------------------------------------------------
```

```
// REFERENCE PRICE (SAFE FALLBACK LOGIC)
// -------------------------------------------------
double referencePrice = 1.0;
if (minAsk > 0.0 && maxBid > 0.0)
   referencePrice = (minAsk + maxBid) / 2.0;
else if (minAsk > 0.0)
   referencePrice = minAsk;
else if (maxBid > 0.0)
   referencePrice = maxBid;

// -------------------------------------------------
// CREATE SIMULATED ORDERS
// -------------------------------------------------
for (int i = 1; i <= 5; ++i)
{
   double amount = 0.1; // wallet-safe amount
   double priceOffset = 0.002 * i; // deterministic variation

   double askPrice = referencePrice * (1.01 + priceOffset);
   double bidPrice = referencePrice * (0.99 - priceOffset);

   std::cout << "\nOrder set " << i << ":\n";
   std::cout << "  Sell check → requires " << amount << " "
         << product.substr(0, product.find('/')) << "\n";
   std::cout << "  Buy check  → requires "
         << (bidPrice * amount) << " "
         << product.substr(product.find('/') + 1) << "\n";

   // ---------------- ASK (SELL) ----------------
   OrderBookEntry ask(
      askPrice,
      amount,
      currentTime,
      product,
      OrderType::ask,
      currentUser
   );

   if (wallet.canFulfillOrder(ask))
   {
      orderBook.insertOrder(ask);
      appendTransactionCSV(currentUser, "ASK", product, askPrice, amount, currentTime);
   }
```

```
        // ---------------- BID (BUY) ----------------
        OrderBookEntry bid(
            bidPrice,
            amount,
            currentTime,
            product,
            OrderType::bid,
            currentUser
        );

        if (wallet.canFulfillOrder(bid))
        {
            orderBook.insertOrder(bid);
            appendTransactionCSV(currentUser, "BID", product, bidPrice, amount, currentTime);
        }
    }

    // ----------------------------------------------------
    // MATCH & EXECUTE TRADES
    // ----------------------------------------------------
    auto sales = orderBook.matchAsksToBids(product, currentTime);

    for (const auto& sale : sales)
    {
        if (sale.username != currentUser)
            continue;

        wallet.processSale(sale);

        if (sale.orderType == OrderType::asksale)
            appendTransactionCSV(currentUser, "ASK_SALE", product, sale.price, sale.amount,
currentTime);
        else if (sale.orderType == OrderType::bidsale)
            appendTransactionCSV(currentUser, "BID_SALE", product, sale.price, sale.amount,
currentTime);
    }
  }

  wallet.saveToCSV("wallet.csv", currentUser);
  std::cout << "\nSimulation complete. Orders executed and wallet updated.\n";
}


// ====================================================
```

```
// OHLC SUMMARY TABLE
// ============================================================
void printOHLCtable(const std::vector<Candlestick>& candles)
{
  if (candles.empty())
  {
     std::cout << "No data available.\n";
     return;
  }

  std::cout << std::left
          << std::setw(12) << "DATE"
          << std::setw(14) << "OPEN"
          << std::setw(14) << "HIGH"
          << std::setw(14) << "LOW"
          << std::setw(14) << "CLOSE"
          << "\n------------------------------------------------------------\n";

  for (const auto& c : candles)
  {
     std::cout << std::left
             << std::setw(12) << c.startTime
             << std::setw(14) << c.open
             << std::setw(14) << c.high
             << std::setw(14) << c.low
             << std::setw(14) << c.close
             << "\n";
  }

  std::cout << "------------------------------------------------------------\n";
}

void MerkelMain::showUserActivitySummary()
{
  std::ifstream file("transactions.csv");
  if (!file.is_open()) {
     std::cout << "No transaction data found.\n";
     return;
  }

  std::string currentUser = userManager.getLoggedInUser().username;
  std::string line;

  int askCount = 0;
```

```
int bidCount = 0;
double totalSpent = 0.0;

std::map<std::string, int> askPerProduct;
std::map<std::string, int> bidPerProduct;

// Skip header
std::getline(file, line);

while (std::getline(file, line))
{
   std::stringstream ss(line);
   std::string user, type, product, priceStr, amountStr, timestamp;

   std::getline(ss, user, ',');
   std::getline(ss, type, ',');
   std::getline(ss, product, ',');
   std::getline(ss, priceStr, ',');
   std::getline(ss, amountStr, ',');
   std::getline(ss, timestamp, ',');

   if (user != currentUser) continue;

   double price = std::stod(priceStr);
   double amount = std::stod(amountStr);

   if (type == "ASK") {
      askCount++;
      askPerProduct[product]++;
   }
   else if (type == "BID") {
      bidCount++;
      bidPerProduct[product]++;
      totalSpent += price * amount;
   }
}

file.close();

// ================= OUTPUT =================
std::cout << "\n=== USER ACTIVITY SUMMARY ===\n";
std::cout << "User: " << currentUser << "\n\n";

std::cout << "Total ASKs placed: " << askCount << "\n";
```

```
    std::cout << "Total BIDs placed: " << bidCount << "\n";
    std::cout << "Total USDT spent (BIDs): " << totalSpent << "\n";

    std::cout << "\n--- Breakdown by Product ---\n";
    for (const auto& p : askPerProduct)
        std::cout << p.first << " | ASKs: " << p.second << "\n";

    for (const auto& p : bidPerProduct)
        std::cout << p.first << " | BIDs: " << p.second << "\n";

    std::cout << "==============================\n\n";
}
```

// ====================== END OF MY OWN CODE ======================

MerkelMain.h:

```
#pragma once

#include <string>
#include <vector>
#include "OrderBook.h"
#include "Wallet.h"
#include "UserManager.h"
#include "Candlestick.h"

class MerkelMain {
public:
    MerkelMain();
    void init();

private:
    // ====================== MENU & CORE ======================
    void printMenu();
    void printHelp();
    void printMarketStats();
    void printProducts();
    void printWallet();
    void mainLoop();
```

```
    void gotoNextTimeFrame();
    void enterAsk();
    void enterBid();
    void showTransactionHistory();
    void printProductSummary();
    void showUserActivitySummary();

// ============================================================
// START OF MY OWN CODE
// ============================================================

    // ====================== MINUTE CANDLESTICKS ======================
    // Build minute-based OHLC data
std::vector<Candlestick> buildMinuteCandlesticks(const std::string& product);
std::vector<OrderBookEntry>
getTransactionOrdersForProduct(const std::string& product);
    // Candlestick table
    void printMinuteCandlestickTable(const std::vector<Candlestick>& candles,
                                     int startIndex, int endIndex);

    // (Graph output removed: coursework requires OHLC table only)

    // Timeframe selection menu
    void handleCandlestickTimeframe(const std::string& product);

    // Synthetic candlestick generator (if CSV has few minutes)
    void generateSyntheticCandles(std::vector<Candlestick>& candles, int targetCount);

    // ====================== DATA MEMBERS ======================
    OrderBook orderBook;
    Wallet wallet;
    std::string currentTime;

    // User authentication
    UserManager userManager = UserManager("users.csv");
    bool userLoggedIn = false;

    // ====================== CSV LOGGING ======================
    void logTransaction(const std::string& type,
                        const std::string& product,
                        double price,
                        double amount);

    // Wallet operations
```

```
    void depositMoney();
    void withdrawMoney();

    // ====================== TRADE SIMULATION ======================
    void simulateTradingActivity();
};

// ====================== END OF MY OWN CODE ======================
```

OrderBook.cpp:

```
#include "OrderBook.h"
#include "CSVReader.h"
#include <set>
#include <algorithm>

OrderBook::OrderBook(const std::string& filename) {
    orders = CSVReader::readCSV(filename);
}

void OrderBook::insertOrder(const OrderBookEntry& order) {
    orders.push_back(order);
}

std::vector<std::string> OrderBook::getKnownProducts() const {
    std::set<std::string> productSet;

    for (const auto& e : orders)
        productSet.insert(e.product);

    return std::vector<std::string>(productSet.begin(), productSet.end());
}

std::string OrderBook::getEarliestTime() const {
    if (orders.empty()) return "";
    std::string earliest = orders[0].timestamp;

    for (const auto& e : orders)
        if (e.timestamp < earliest)
            earliest = e.timestamp;

    return earliest;
}
```

```
std::string OrderBook::getNextTime(const std::string& currentTime) const {
    std::string nextTime;

    for (const auto& e : orders) {
        if (e.timestamp > currentTime) {
            if (nextTime.empty() || e.timestamp < nextTime)
                nextTime = e.timestamp;
        }
    }

    if (nextTime.empty())
        return getEarliestTime();

    return nextTime;
}

std::vector<OrderBookEntry> OrderBook::getOrders(
    const std::string& product,
    const std::string& timestamp,
    OrderType orderType
) const {
    std::vector<OrderBookEntry> result;

    for (const auto& e : orders) {
        if (e.product == product &&
            e.timestamp == timestamp &&
            e.orderType == orderType) {
            result.push_back(e);
        }
    }

    return result;
}

// ============================================================
// START OF MY OWN CODE
// ============================================================
std::vector<OrderBookEntry> OrderBook::getAllOrdersForProduct(
    const std::string& product
) const {
    std::vector<OrderBookEntry> result;

    for (const auto& e : orders)
```

```
        if (e.product == product)
            result.push_back(e);

    // sort by timestamp for time-based aggregation
    std::sort(result.begin(), result.end(),
              [](const OrderBookEntry& a, const OrderBookEntry& b){
                  return a.timestamp < b.timestamp;
              });

    return result;
}
// ====================== END OF MY OWN CODE ======================


// ============================================================
// START OF MY OWN CODE
// ============================================================
std::vector<OrderBookEntry> OrderBook::matchAsksToBids(
    const std::string& product,
    const std::string& timestamp
) const {
    std::vector<OrderBookEntry> sales;

    std::vector<OrderBookEntry> asks;
    std::vector<OrderBookEntry> bids;

    for (const auto& e : orders) {
        if (e.product == product && e.timestamp == timestamp) {
            if (e.orderType == OrderType::ask)
                asks.push_back(e);
            else if (e.orderType == OrderType::bid)
                bids.push_back(e);
        }
    }

    if (asks.empty() || bids.empty())
        return sales;

    std::sort(asks.begin(), asks.end(),
              [](const OrderBookEntry& a, const OrderBookEntry& b){
                  return a.price < b.price;
              });

    std::sort(bids.begin(), bids.end(),
```

```
        [](const OrderBookEntry& a, const OrderBookEntry& b){
            return a.price > b.price;
        });

for (auto& ask : asks) {
    for (auto& bid : bids) {
        if (bid.amount <= 0) continue;
        if (ask.amount <= 0) break;

        if (bid.price >= ask.price) {
            double tradedAmount =
                std::min(ask.amount, bid.amount);
            if (tradedAmount <= 0) continue;

            double tradePrice = ask.price;

            OrderBookEntry saleSeller{
                tradePrice,
                tradedAmount,
                timestamp,
                product,
                OrderType::asksale,
                ask.username
            };

            OrderBookEntry saleBuyer{
                tradePrice,
                tradedAmount,
                timestamp,
                product,
                OrderType::bidsale,
                bid.username
            };

            sales.push_back(saleSeller);
            sales.push_back(saleBuyer);

            ask.amount -= tradedAmount;
            bid.amount -= tradedAmount;
        }
    }
}

return sales;
```

```
}
// ====================== END OF MY OWN CODE ======================
```

OrderBook.h:

```
#pragma once

#include <vector>
#include <string>
#include "OrderBookEntry.h"

class OrderBook {
public:
  // Load all orders from CSV in constructor
  OrderBook(const std::string& filename);

  // Add new order
  void insertOrder(const OrderBookEntry& order);

  // List of all products
  std::vector<std::string> getKnownProducts() const;

  // Earliest timestamp
  std::string getEarliestTime() const;

  // Next timestamp after currentTime
  std::string getNextTime(const std::string& currentTime) const;

  // Orders matching exact timestamp + product + type
  std::vector<OrderBookEntry> getOrders(
     const std::string& product,
     const std::string& timestamp,
     OrderType orderType
  ) const;

  // NEW: Get ALL orders in the CSV for this product
  std::vector<OrderBookEntry> getAllOrdersForProduct(const std::string& product) const;

  // Match engine
  std::vector<OrderBookEntry> matchAsksToBids(
     const std::string& product,
     const std::string& timestamp
  ) const;
```

```
private:
  std::vector<OrderBookEntry> orders;
};
```

OrderBookEntry.cpp:

```
#include "OrderBookEntry.h"

OrderBookEntry::OrderBookEntry(double _price,
                       double _amount,
                       const std::string& _timestamp,
                       const std::string& _product,
                       OrderType _orderType,
                       const std::string& _username)
  : price{_price},
    amount{_amount},
    timestamp{_timestamp},
    product{_product},
    orderType{_orderType},
    username{_username} {}
```

OrderBookEntry.h:

```
#pragma once
#include <string>

enum class OrderType {
  bid,
  ask,
  asksale,
  bidsale
};

class OrderBookEntry {
public:
  double price;
  double amount;
  std::string timestamp;
  std::string product;
  OrderType orderType;
  std::string username;
```

```
  OrderBookEntry(double price,
                 double amount,
                 const std::string& timestamp,
                 const std::string& product,
                 OrderType orderType,
                 const std::string& username = "dataset");
};
```

User.cpp:

```
#include "User.h"
```

User.h:

```
// START OF MY OWN CODE
#pragma once
#include <string>

class User {
public:
  std::string username;
  std::string fullname;
  std::string email;
  size_t passwordHash;

  User() {}
  User(std::string uname, std::string fname, std::string mail, size_t pHash)
     : username(uname), fullname(fname), email(mail), passwordHash(pHash) {}
};

// ====================== END OF MY OWN CODE ======================
```

UserManager.cpp:

```
// START OF MY OWN CODE
#include "UserManager.h"
#include "User.h"
#include "Wallet.h"

#include <fstream>
```

```
#include <sstream>
#include <iostream>
#include <functional>
#include <vector>
#include <cstdlib>   // rand
#include <ctime>     // time

// Constructor
UserManager::UserManager(const std::string& filename)
  : filename(filename), logged(false)
{
  // Seed RNG once
  static bool seeded = false;
  if (!seeded) {
     srand(static_cast<unsigned>(time(nullptr)));
     seeded = true;
  }

  // Ensure CSV has a header
  std::ifstream check(filename);
  if (!check.good()) {
     std::ofstream file(filename);
     file << "username,fullname,email,passwordHash\n";
  }
}

// --------------------
// Load all users
// --------------------
std::vector<User> UserManager::loadUsers()
{
  std::vector<User> users;
  std::ifstream file(filename);
  if (!file.is_open()) return users;

  std::string line;
  std::getline(file, line); // skip header

  while (std::getline(file, line)) {
     std::stringstream ss(line);
     std::string username, fullname, email, hashStr;

     std::getline(ss, username, ',');
     std::getline(ss, fullname, ',');
```

```
        std::getline(ss, email, ',');
        std::getline(ss, hashStr, ',');

        if (username.empty()) continue;

        size_t hash = std::stoull(hashStr);
        users.emplace_back(username, fullname, email, hash);
    }
    return users;
}

// --------------------
// Save user
// --------------------
void UserManager::saveUser(const User& user)
{
    std::ofstream file(filename, std::ios::app);
    if (!file.is_open()) return;

    file << user.username << ","
         << user.fullname << ","
         << user.email << ","
         << user.passwordHash << "\n";
}

// --------------------
// Check duplicate fullname+email
// --------------------
bool UserManager::userExists(const std::string& fullname,
                             const std::string& email)
{
    auto users = loadUsers();
    for (const auto& u : users) {
        if (u.fullname == fullname && u.email == email)
            return true;
    }
    return false;
}

// --------------------
// Check username uniqueness
// --------------------
bool UserManager::usernameExists(const std::string& username)
{
```

```
    auto users = loadUsers();
    for (const auto& u : users) {
        if (u.username == username)
            return true;
    }
    return false;
}

// --------------------
// Generate UNIQUE 10-digit username
// --------------------
std::string UserManager::generateUserID()
{
    std::string id;
    do {
        long long num =
            1000000000LL + (rand() % 9000000000LL); // 10 digits
        id = std::to_string(num);
    } while (usernameExists(id));

    return id;
}

// --------------------
// Register user
// --------------------
bool UserManager::registerUser(const std::string& fullname,
                               const std::string& email,
                               const std::string& password)
{
    if (!isValidGmail(email)) {
        std::cout << "Registration failed: email must end with @gmail.com\n";
        return false;
    }

    if (userExists(fullname, email)) {
        std::cout << "User already exists.\n";
        return false;
    }

    std::hash<std::string> hasher;
    size_t hashed = hasher(password);

    std::string username = generateUserID();
```

```
    User newUser(username, fullname, email, hashed);

    saveUser(newUser);

    // CREATE INITIAL WALLET HERE (ONLY ONCE)
    Wallet w;
    w.insertCurrency("BTC", 10);
    w.insertCurrency("USDT", 1000);
    w.saveToCSV("wallet.csv", username);

    loggedInUser = newUser;
    logged = true;

    std::cout << "\nRegistration successful!\n";
    std::cout << "Your username: " << username << "\n\n";

    return true;
}


// --------------------
// Login user
// --------------------
bool UserManager::loginUser(const std::string& username,
                            const std::string& password)
{
    auto users = loadUsers();
    std::hash<std::string> hasher;
    size_t hashedInput = hasher(password);

    for (const auto& u : users) {
        if (u.username == username &&
            u.passwordHash == hashedInput)
        {
            loggedInUser = u;
            logged = true;

            std::cout << "\nLogin successful!\n";
            std::cout << "Welcome, " << u.fullname << "\n\n";
            return true;
        }
    }

    std::cout << "Invalid username or password.\n";
```

```
  return false;
}

// --------------------
// Accessors
// --------------------
User UserManager::getLoggedInUser()
{
  return loggedInUser;
}

bool UserManager::isLoggedIn()
{
  return logged;
}

bool UserManager::resetPassword(const std::string& username,
                                const std::string& email,
                                const std::string& newPassword)
{
  // Enforce Gmail-only email for password reset
if (!isValidGmail(email)) {
  std::cout << "Password reset failed: email must end with @gmail.com\n";
  return false;
}

  auto users = loadUsers();
  bool found = false;

  std::hash<std::string> hasher;
  size_t newHash = hasher(newPassword);

  // Rewrite CSV safely
  std::ofstream out(filename);
  if (!out.is_open()) {
     std::cout << "ERROR: Unable to update users file.\n";
     return false;
  }

  // Write header
  out << "username,fullname,email,passwordHash\n";

  for (auto& u : users)
  {
```

```
        if (u.username == username && u.email == email)
        {
            u.passwordHash = newHash;
            found = true;
        }

        out << u.username << ","
            << u.fullname << ","
            << u.email << ","
            << u.passwordHash << "\n";
    }

    out.close();

    if (!found)
    {
        std::cout << "Password reset failed: user not found or email mismatch.\n";
        return false;
    }

    std::cout << "Password reset successful. You may now log in.\n";
    return true;
}

std::vector<std::string> UserManager::getAllUsernames()
{
    std::vector<std::string> names;
    auto users = loadUsers();

    for (const auto& u : users)
        names.push_back(u.username);

    return names;
}

bool UserManager::isValidGmail(const std::string& email)
{
    const std::string suffix = "@gmail.com";
    if (email.length() < suffix.length()) return false;

    return email.substr(email.length() - suffix.length()) == suffix;
}

// ========== END OF MY OWN CODE ==========
```

UserManager.h:

```
// ========================================================
// START OF MY OWN CODE
// ========================================================

#pragma once
#include <vector>
#include <string>
#include "User.h"

class UserManager {
public:
  UserManager(const std::string& filename);

  bool registerUser(const std::string& fullname,
                    const std::string& email,
                    const std::string& password);

  bool loginUser(const std::string& username,
                 const std::string& password);

  User getLoggedInUser();
  bool isLoggedIn();
  bool resetPassword(const std::string& username,
                 const std::string& email,
                 const std::string& newPassword);
std::vector<std::string> getAllUsernames();


private:
  std::string filename;
  User loggedInUser;
  bool logged = false;

  std::vector<User> loadUsers();
  void saveUser(const User& user);
  bool userExists(const std::string& fullname, const std::string& email);
  std::string generateUserID();
bool usernameExists(const std::string& username);
bool isValidGmail(const std::string& email);
```

```
};

// ====================== END OF MY OWN CODE ======================
```

Wallet.cpp:

```
#include "Wallet.h"
#include "CSVReader.h"

#include <sstream>
#include <fstream>
#include <vector>
#include <iostream>

// ============================================================
// START OF MY OWN CODE
// ============================================================

// INTERNAL HELPER
static void ensureWalletCSV(const std::string& filename)
{
    std::ifstream in(filename);
    if (!in.good())
    {
        std::ofstream out(filename);
        out << "username,currency,amount\n";
        return;
    }

    in.seekg(0, std::ios::end);
    if (in.tellg() == 0)
    {
        std::ofstream out(filename);
        out << "username,currency,amount\n";
    }
}

// ====================== END OF MY OWN CODE ======================

void Wallet::insertCurrency(const std::string& type, double amount)
{
    if (amount < 0) return;
```

```
    currencies[type] += amount;
}

bool Wallet::containsCurrency(const std::string& type, double amount) const
{
    if (amount < 0) return false;

    auto it = currencies.find(type);
    if (it == currencies.end()) return false;

    return it->second >= amount;
}

bool Wallet::removeCurrency(const std::string& type, double amount)
{
    if (amount < 0) return false;

    auto it = currencies.find(type);
    if (it == currencies.end()) return false;
    if (it->second < amount) return false;

    it->second -= amount;
    return true;
}

std::string Wallet::toString() const
{
    std::ostringstream oss;
    oss << "Wallet contents:\n";

    if (currencies.empty()) {
        oss << "  (empty)\n";
        return oss.str();
    }

    for (const auto& pair : currencies)
        oss << "  " << pair.first << " : " << pair.second << "\n";

    return oss.str();
}

bool Wallet::canFulfillOrder(const OrderBookEntry& order) const
{
    auto tokens = CSVReader::tokenize(order.product, '/');
```

```
    if (tokens.size() != 2) return false;

    const std::string& base  = tokens[0];
    const std::string& quote = tokens[1];

    if (order.orderType == OrderType::ask)
        return containsCurrency(base, order.amount);

    if (order.orderType == OrderType::bid)
        return containsCurrency(quote, order.price * order.amount);

    return false;
}

void Wallet::processSale(const OrderBookEntry& sale)
{
    auto tokens = CSVReader::tokenize(sale.product, '/');
    if (tokens.size() != 2) return;

    const std::string& base  = tokens[0];
    const std::string& quote = tokens[1];

    if (sale.orderType == OrderType::asksale)
    {
        removeCurrency(base, sale.amount);
        insertCurrency(quote, sale.amount * sale.price);
    }
    else if (sale.orderType == OrderType::bidsale)
    {
        removeCurrency(quote, sale.amount * sale.price);
        insertCurrency(base, sale.amount);
    }
}

// ============================================================
// START OF MY OWN CODE
// ============================================================
void Wallet::loadFromCSV(const std::string& filename,
                         const std::string& username)
{
    currencies.clear();

    std::ifstream file(filename);
    if (!file.is_open()) return;
```

```
    std::string line;
    std::getline(file, line); // skip header

    while (std::getline(file, line))
    {
        std::stringstream ss(line);
        std::string user, currency, amountStr;

        std::getline(ss, user, ',');
        std::getline(ss, currency, ',');
        std::getline(ss, amountStr, ',');

        if (user == username)
            currencies[currency] = std::stod(amountStr);
    }
}

void Wallet::saveToCSV(const std::string& filename,
                       const std::string& username)
{
    ensureWalletCSV(filename);

    std::vector<std::string> lines;
    std::ifstream in(filename);

    // Keep all rows EXCEPT this user's
    if (in.is_open())
    {
        std::string line;
        std::getline(in, line); // skip header

        while (std::getline(in, line))
        {
            if (line.rfind(username + ",", 0) != 0)
                lines.push_back(line);
        }
        in.close();
    }

    std::ofstream out(filename);
    out << "username,currency,amount\n";

    for (const auto& l : lines)
```

```
        out << l << "\n";

    for (const auto& pair : currencies)
    {
        out << username << ","
            << pair.first << ","
            << pair.second << "\n";
    }
}

bool Wallet::existsInCSV(const std::string& filename,
                         const std::string& username)
{
    std::ifstream file(filename);
    if (!file.is_open()) return false;

    std::string line;
    while (std::getline(file, line))
    {
        if (line.rfind(username + ",", 0) == 0)
            return true;
    }
    return false;
}
// ====================== END OF MY OWN CODE ======================
```

Wallet.h:

```
#pragma once

#include <map>
#include <string>
#include "OrderBookEntry.h"

class Wallet {
public:
    Wallet() = default;

    // Add some amount of a currency to the wallet
```

```
    void insertCurrency(const std::string& type, double amount);

    // Check if wallet has at least this amount of currency
    bool containsCurrency(const std::string& type, double amount) const;

    // Try to remove amount from wallet. Return true on success.
    bool removeCurrency(const std::string& type, double amount);

    bool existsInCSV(const std::string& filename,
                     const std::string& username);


    // Return a human-readable string summarising wallet contents
    std::string toString() const;

    // Check if the wallet can afford to place this order
    bool canFulfillOrder(const OrderBookEntry& order) const;

    // Update wallet after a sale (asksale or bidsale)
    void processSale(const OrderBookEntry& sale);

    // Load wallet data for a specific user from CSV
    void loadFromCSV(const std::string& filename,
                     const std::string& username);

    // Save wallet data for a specific user to CSV
    void saveToCSV(const std::string& filename,
                   const std::string& username);

private:
    std::map<std::string, double> currencies;
};
```